Tell

Every contract has a tell.

A privacy leak scanner for Midnight. It reads a Compact contract the way the compiler does and shows, in bits, what each circuit gives away.

`tell-midnight.vercel.app · github.com/RohanGlitched/tell`

```
public ledger state

ballots[fc3aefaa…] = e73f9302…

voted Chen

ballots[6ba7b2d2…] = bd4b2304…

voted Bilal

ballots[ea78c8e1…] = c81b9533…

voted Asha
```

Four candidates, four guesses: every vote in this “private” ballot reads back from public state.

Proofs hide the inputs. The transcript does not.

Every Midnight transaction carries a public transcript: each ledger read and write, with its operands. A zero-knowledge proof keeps the private inputs off the chain, but whatever the contract writes there is public forever.

So a contract can be zero-knowledge and still give its secrets away, one innocent-looking line at a time.

```
ballots.insert(tag,
disclose(persistentHash(choice)));
```

The vote is hidden behind a hash. But there are only four candidates.

4 guesses → every vote

The compiler asks whether. Tell measures how much.

This line	Compiler	Tell
`disclose(hash(vote))`	accepted	2 bits once asserted below four; four guesses recover it
`disclose(hash(id, salary))`	accepted	id is public; salary falls to brute force
`if (disclose(bid ≥ reserve)) a.insert(t) else b.insert(t)`	accepted	which set was written shows the bid's side
`posts.insert(n, disclose(hash(key)))`	accepted	same tag on every post: one identified post unmasks all
`owner = disclose(degradeToTransient(sk))`	accepted	a cast, not a hash: the key is on the ledger

Compact's **disclose()** is a yes/no gate. Tell starts where it stops, on the compiler's own output, so it complements the compiler and could run as one of its passes.

We put every Midnight Buildathon app under the lamp.

96

projects analyzed, 248 contracts compiled with the compiler each one asks for

46%

publish a fixed tag that can tie a user's transactions together, usually by design

15

have a finding at medium or worse; one critical: exact GPS locations behind an unsalted hash

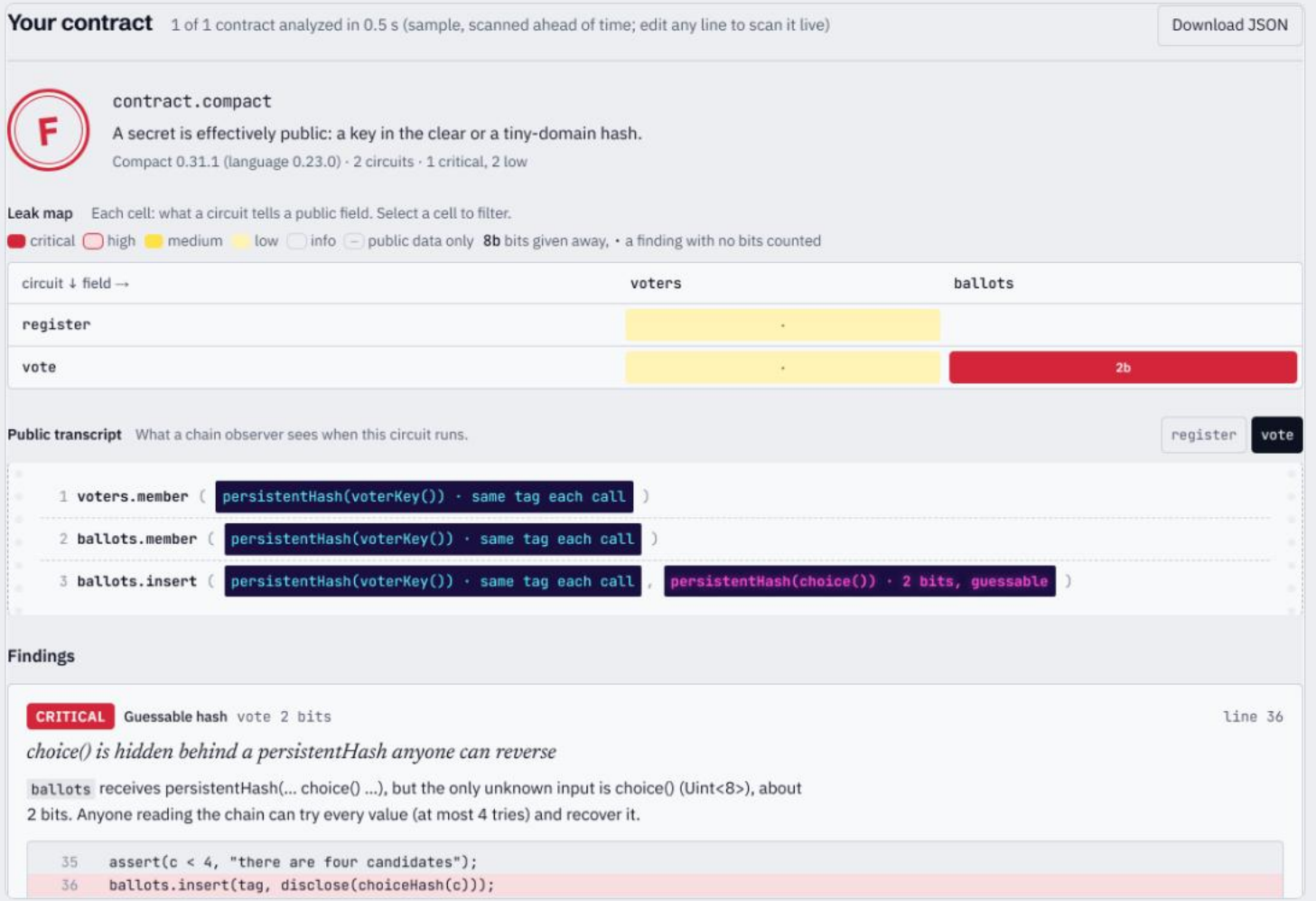


Grades by project; F: 1 (red). Every medium-or-worse finding was read by hand against the source. No team is named; findings are for the team concerned.

Paste a contract. See what it tells.

- A grade, A to F
- A leak map: every circuit against every public field, in bits
- The public transcript, operand by operand
- Each finding with its line and its fix

`tell-midnight.vercel.app/scan`



How Tell reads a contract

1

The real compiler

compactc, matched to the contract's pragma: 0.29 to 0.35, preferring 0.31.1, the ledger live today.

2

Its own circuit form

reduce-to-circuit: helpers inlined, loops unrolled, every step a guarded binding.

3

Information flow

Every witness and argument traced through hashes, casts, comparisons and branches, counting the unknown bits that survive.

4

Grade and report

Text, JSON or SARIF, with line, fix and the public transcript of each circuit.

Not a re-implementation of Compact: Tell reads exactly what the compiler builds, so it sees the same circuits the proofs are made from.

Every claim has a test that attacks it

5 + 5

The leak zoo

Five leaky contracts and their fixes: a ballot, a salary record, a sealed sale, a staff board, a vault.

10

Attack tests

Each reads the secret back out of public ledger state on the leaky contract, and fails to on the fix.

87

Precision tests

Every zoo leak found once, nothing above low on the fixes, evasion cases from review, Midnight's own examples, two compiler output formats.

All of it runs in CI on every push, and the zoo is re-checked contract by contract under compilers 0.30, 0.31, 0.34 and 0.35.

Same votes. Two contracts. One gives them away.

`ballot-leaky`

Stores hash(choice). The site reads its public state from Midnight's indexer, hashes the four candidates and matches every ballot.

`every vote read back`

`ballot-fixed`

Stores a salted commitment. The same attack, on the same votes, matches nothing.

`no match in 4 guesses`

Deployed on Midnight Preprod by Tell's own house wallet, proved in WASM with Midnight's SDK. tell-midnight.vercel.app/live

Who it protects

Midnight developers

Findings land on the pull request, before a contract is deployed and its history is public forever.

Auditors and reviewers

A map of every disclosure, in bits, per circuit and field: where to look first, and evidence for the report.

The people in these apps

Votes, salaries, locations, identities: what these contracts hold. They never see Tell; they just stop being readable.

Tell itself needs no private data. It reads source code and public state only, and the survey is published under salted numbers so no team can be identified from it.

Three ways to run it, no setup for the first

In the browser

Paste a contract or a public GitHub repository at tell-midnight.vercel.app/scan. The real compiler runs on every scan.

In CI

```
- uses: RohanGlitched/tell@v1
with:
path: contract
fail-on: high
```

SARIF lands in GitHub code scanning, on the line that leaks.

On the command line

```
npx tsx engine/src/cli.ts .
--sarif tell.sarif
--fail-on high
```

Intended disclosures are marked once: // tell:allow published

Free core. Paid for the way analyzers are.

Slither and Aderyn live on audit work, Certora on DAO grants, the Move Prover inside its chain's toolchain. Hosted-only MythX was shut down.

1. **Tooling grant**: rule research, every new compiler release
2. **Audit firms**: the privacy section of a Compact review
3. **The compiler**: running as one of its passes is the goal
4. **Hosted checks, later**: private repos only; public stays free

Now

Scanner, CLI, GitHub Action with SARIF, leak zoo, survey of every buildathon app, live demo on Preprod

Wave 3

Pull request comments with the fix inline; constructor analysis; shielded token and DUST patterns

Next

A VS Code extension that marks leaking lines as you type; a Tell check in Midnight's project templates

Later

History per contract, team dashboards, a privacy report auditors can attach

Find your contract's tell before the chain does.

Make Tell part of the Midnight developer path: the docs, the project templates, and every pull request that touches a contract.

tell-midnight.vercel.app

github.com/RohanGlitched/tell